

PackageTriage

How to use it

Five steps: drop the .dtsx package, open the queue, decide, handle the manual work, rebuild.

1 · Run the pipeline

A .dtsx package in — a full analysis package out.

Inputs

a .dtsx file or a directory of packages — adjacent .conmgr files are read to resolve project-level connections

Command

```
python pipeline.py <package.dtsx> <outdir>
```

What you get

- *_canonical.json — the normalized task/edge graph
- *_decisions.json — the work queue (stable ids)
- *_dag.md · *_dag.png — layered precedence diagram
- *_review.md · *_hitl.xlsx · *_run_report.json

Parsing

executables, precedence constraints, data-flow components, SQL statements and connection refs come straight from the XML; credentials are masked before parsing, packages never run

2 · Open the review workspace

A local, zero-dependency web client — runs where the data lives.

Start

```
python server.py <outdir>  
-> http://localhost:8787  
same URL for everyone — locally or via container
```

The queue

every warn flag and dropped item is a numbered decision.
Header shows progress: N of M decided — flags ·
assumptions · dropped. Info findings sit below as context.

Click an item

see the canonical source slice and the DAG node it lands on
— cycles show the full A -> B -> ... -> A precedence path

The graph view

layered precedence order with flag tinting — orphan tasks,
cycles and external integrations are visible at a glance

3 · Decide — in verbs, not statuses

The button says what happens to the graph, not what box is ticked.

circular_flow	Break cycle — drop first edge · Keep — intentional retrigger
script_component	Manual rewrite planned
expression_driven	Map expressions to target
conditional_precedence	Map to target conditions
project_connection	Connection migrated — approve
external_integration	Has equivalent — approve · Drop task
exec_package	Resolve child package — approve
loop_container / event_handler	Iteration / handler mapped — approve
dynamic_sql	Documented — will map manually
disabled_task	Stays disabled · Drop task
orphan_task	Entry point — keep · Dead task — drop
not_migrated	Acknowledge — stays dropped

Custom override... always available (whitelisted canonical fields).

ignore on a task or precedence edge strips it before re-analysis

4 · Manual work — the honest backlog

Everything the engine can't produce itself, counted in the header.

Script tasks

embedded .NET code — the logic must be read and rewritten in the target platform

Expressions

runtime-driven values and conditional precedence — each expression needs an explicit mapping

Project connections

connection strings live in .conmgr/.dtproj files — migrate the configuration, not just the package

External integrations

FTP, child packages, OS tasks — each needs an orchestrator equivalent in the target

5 · Rebuild — and done

Apply decisions -> rebuild artifacts. ~1 second per cycle.

What rebuilds

dag.png, dag.md, review.md, hitl.xlsx and decisions.json
(merged — your decisions persist)

What changed

ignored tasks and precedence edges are gone, cycles visibly break, stale decisions are marked — the diff is the work

Done means

queue empty AND manual work ~0: a reproducible spec
(canonical + decisions.json) and a flow you can port from

Prefer Excel?

fill the Decision column in *_hitl.xlsx (ok / reject /
override field=value) and run: decisions.py import <xlsx>